· A data structure (DS) is a concrete implementation of the specification provided by an ADT.

You can think about an ADT as the blueprint, while a DS is the translation of those specifications into real code.
An ADT is defined from the point of view of who uses it, by describing its behavior in terms of possible values, possible operations on it, and the output and side effects of these operations.
A more formal definition would describe an ADT as a set of types, a designated type from that type set, a set of functions, and a set of axioms.
In contrast, a data structure, which is a concrete representation of data, is described from the point of view of an implementer, not a user.

Back to our array example, a possible ADT for a static array is, for example: “an array is a container that can store a fixed number of elements, each associated with an index (the position of the element within the array), and access any element by its position (random access)”.

Its implementation, however, needs to take care of details like:

· Will the array size be fixed at creation or can it be modified?
· Will the array be allocated statically or dynamically?
· Will the array host only elements of a single type, or of any type?